

Báo cáo Chương 12

Mã hóa khóa công khai và RSA

Theory, security và các tấn công trong thực tế

Chủ đề học phần: Chương 12 — Mã hóa khóa công khai và RSA

Ngày soạn: June 22, 2026

Contents

1	Giới thiệu về Mã hóa khóa công khai	3
2	Cơ sở toán học	4
2.1	Số học đồng dư	5
2.2	Hàm phi Euler	5
2.3	Định lý Euler	6
2.4	Bài toán logarit rời rạc	6
2.5	Giả thuyết Diffie–Hellman tính toán	7
2.6	Giả thuyết Diffie–Hellman quyết định	7
2.7	Giả thuyết phân tích thừa số	7
3	ElGamal Cryptosystem	8
3.1	Sinh khóa	8
3.2	Mã hóa	9
3.3	Giải mã	9
3.4	Phân tích an toàn	10
3.5	Ví dụ tính toán nhỏ	10
4	RSA Trapdoor Permutation	11
4.1	Sinh khóa	11
4.2	Mã hóa	12
4.3	Giải mã	12
4.4	Giải thích trapdoor permutation	13
4.5	Chứng minh tính đúng đắn	13
5	Vì sao Textbook RSA không an toàn	14
5.1	Mã hóa tất định	14
5.2	Tấn công thông điệp nhỏ	14
5.3	Tấn công dựa trên tính nhân	15
5.4	Ví dụ tấn công trong bài giảng	15

6	RSA Padding và mã hóa RSA an toàn	16
6.1	Vì sao cần đệm	16
6.2	PKCS#1 v1.5	17
6.3	OAEP	18
7	Tình huống tấn công thực tế: Tấn công Bleichenbacher	19
7.1	Bối cảnh	19
7.2	Padding oracle	19
7.3	Diễn tiến tấn công	20
7.4	Vì sao điều này quan trọng	21
8	Kết luận	21

1. Giới thiệu về Mã hóa khóa công khai

Mã hóa khóa công khai ra đời nhằm giải quyết một hạn chế cơ bản của mã hóa đối xứng: **phân phối khóa bí mật ở quy mô lớn**. Trong một hệ đối xứng, hai bên phải cùng chia sẻ cùng một khóa bí mật trước khi có thể bắt đầu liên lạc bảo mật. Yêu cầu này có thể chấp nhận được trong môi trường nhỏ và được kiểm soát chặt chẽ, nhưng nó trở nên ngày càng khó khăn khi số lượng đối tượng liên lạc tăng lên. Nếu một mạng có n người dùng và mỗi cặp người dùng cần một khóa đối xứng riêng biệt, thì tổng số khóa cần quản lý là

$$\frac{n(n-1)}{2}.$$

Sự tăng trưởng theo bậc hai này tạo ra gánh nặng vận hành rất lớn, bởi vì mỗi khóa bí mật đều phải được sinh ra, trao đổi, lưu trữ và thay mới định kỳ thông qua một kênh an toàn.

Bài toán phân phối khóa không chỉ là một bất tiện; nó là một trở ngại mang tính cấu trúc đối với liên lạc bảo mật quy mô lớn. Nếu quá trình trao đổi khóa đối xứng ban đầu bị chặn bắt hoặc bị can thiệp, tính bí mật của toàn bộ các thông điệp về sau sẽ bị phá vỡ. Vì vậy, độ an toàn của hệ thống phụ thuộc vào việc thiết lập bí mật trước cả khi mã hóa bắt đầu. Mã hóa khóa công khai đưa ra một mô hình khác, giảm bớt sự phụ thuộc vào việc chia sẻ trước một khóa bí mật.

Trong bối cảnh khóa công khai, mỗi người dùng sinh ra một cặp khóa liên hệ với nhau về mặt toán học: **khóa công khai**, có thể được phân phối rộng rãi, và **khóa bí mật**, phải được giữ kín. Ký hiệu khóa công khai là pk và khóa bí mật là sk . Mô hình mã hóa tổng quát có thể viết là

$$c = \text{Enc}_{pk}(m)$$

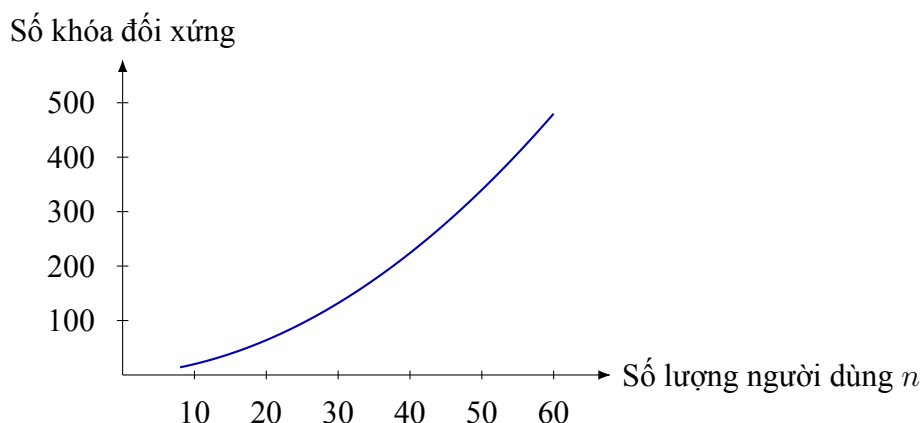
để mã hóa thông điệp m thành bản mã c , và

$$m = \text{Dec}_{sk}(c)$$

để giải mã bằng khóa bí mật tương ứng. Ý tưởng cốt lõi là bất kỳ ai cũng có thể mã hóa thông điệp bằng khóa công khai, nhưng chỉ người nắm giữ khóa bí mật mới có thể khôi phục bản rõ một cách hiệu quả. Sự tách biệt giữa khóa mã hóa được công khai và khóa giải mã được giữ kín chính là bước tiến khái niệm trung tâm của mã hóa khóa công khai.

Ý nghĩa của mô hình này thể hiện ở hai điểm. Thứ nhất, nó đơn giản hóa liên lạc bảo mật giữa các bên chưa từng gặp nhau hoặc chưa từng trao đổi trước bí mật. Người gửi chỉ cần có khóa công khai của người nhận là đã có thể mã hóa thông điệp một cách an toàn. Thứ hai, nó đặt nền tảng cho **các hệ thống liên lạc bảo mật hiện đại trên mạng mở**. Mặc dù các giao thức thực tế thường kết hợp kỹ thuật khóa công khai và đối xứng để đạt hiệu năng cao, khuôn khổ khóa công khai giải quyết bài toán thiết lập niềm tin và khóa ban đầu mà mã hóa đối xứng đơn lẻ không xử lý thuận tiện.

Xét về mặt khái niệm, mã hóa khóa công khai chuyển câu hỏi từ “Làm thế nào để hai bên bí mật chia sẻ cùng một khóa?” thành “Làm thế nào để một bên công bố đủ thông tin cho phép mã



Tăng trưởng bậc hai: trong một mạng mã hóa đối xứng có n người dùng, số khóa chia sẻ phân biệt là

$$\frac{n(n-1)}{2}.$$

Khi số người dùng tăng lên, hệ thống phải quản lý nhiều khóa bí mật hơn một cách rất nhanh.

Quá tải vận hành: mỗi khóa bổ sung đều phải được sinh ra, trao đổi an toàn, lưu trữ, bảo vệ và cuối cùng là luân chuyển hoặc thay thế.



Mô hình khóa công khai: một khóa công khai có thể được phân phối công khai, trong khi chỉ khóa bí mật sk phải được giữ kín.

Figure 1: Mô hình đối xứng mở rộng kém do số khóa chia sẻ tăng theo bậc hai, trong khi mô hình khóa công khai tách biệt việc phân phối công khai khỏi quyền giải mã bí mật.

hóa mà vẫn không lộ khả năng giải mã?” Sự chuyển đổi này dẫn trực tiếp đến việc nghiên cứu các cấu trúc dựa trên lý thuyết số như ElGamal và RSA, trong đó an toàn được xây dựng trên **các giả thuyết độ khó tính toán** thay vì sự bí mật của một kênh chia sẻ từ trước. Vì lý do đó, mã hóa khóa công khai là một chủ đề nền tảng của mật mã hiện đại và là điểm khởi đầu thiết yếu để hiểu các hệ thống được trình bày trong Chương 12.

2. Cơ sở toán học

Những hệ mã được trình bày trong báo cáo này dựa trên một tập hợp gọn các ý tưởng toán học thuộc lý thuyết số và độ khó tính toán. Phần này chỉ trình bày những kiến thức cần thiết để hiểu ElGamal và RSA. Mục tiêu là giới thiệu các khái niệm hỗ trợ trực tiếp cho lập luận bảo mật và chứng minh tính đúng đắn sẽ được sử dụng ở các phần sau.

2.1 Số học đồng dư

Số học đồng dư nghiên cứu các số nguyên theo một mô-đun cố định $n \geq 2$. Với hai số nguyên a và b , ta viết

$$a \equiv b \pmod{n}$$

khi n chia hết cho $a - b$. Nói cách khác, a và b cho cùng một số dư khi chia cho n . Tính toán theo mô-đun n cho phép quy các giá trị lớn về một tập hữu hạn các lớp số dư

$$\{0, 1, 2, \dots, n-1\}.$$

Cấu trúc này là nền tảng trong mật mã học bởi vì các thuật toán mã hóa và giải mã thường liên quan đến phép nhân lặp lại và lũy thừa theo mô-đun một số nguyên lớn. Vì vậy, **số học đồng dư** là ngôn ngữ cơ bản để mô tả cả ElGamal lẫn RSA.

Ví dụ, nếu $17 \equiv 5 \pmod{12}$, thì hai số này đại diện cho cùng một lớp số dư modulo 12. Tương tự, các phép toán đại số được bảo toàn dưới quan hệ đồng dư. Nếu

$$a \equiv b \pmod{n} \quad \text{và} \quad c \equiv d \pmod{n},$$

thì

$$a + c \equiv b + d \pmod{n}$$

và

$$ac \equiv bd \pmod{n}.$$

Tính chất này làm cho số học đồng dư đặc biệt phù hợp với tính toán thuật toán trong các cấu trúc mật mã.

2.2 Hàm phi Euler

Hàm phi Euler, ký hiệu $\varphi(N)$, đếm số lượng số nguyên trong tập $\{1, 2, \dots, N-1\}$ nguyên tố cùng nhau với N . Hàm này đóng vai trò trung tâm trong RSA. Trong trường hợp quan trọng khi

$$N = pq$$

với p và q là hai số nguyên tố phân biệt, giá trị của hàm phi là

$$\varphi(N) = (p-1)(q-1).$$

Lý do là trong các số từ 1 đến $N-1$, những số không nguyên tố cùng nhau với N chính là những số chia hết cho p hoặc q .

Công thức này là then chốt vì **quá trình sinh khóa RSA** phụ thuộc vào việc chọn số mũ công khai e và số mũ bí mật d sao cho

$$ed \equiv 1 \pmod{\varphi(N)}.$$

Sự tồn tại của nghịch đảo modulo này đòi hỏi

$$\gcd(e, \varphi(N)) = 1.$$

2.3 Định lý Euler

Định lý Euler là kết quả cơ bản nối giữa phép lũy thừa modulo và hàm phi. Định lý phát biểu rằng nếu

$$\gcd(x, N) = 1,$$

thì

$$x^{\varphi(N)} \equiv 1 \pmod{N}.$$

Định lý này mở rộng định lý nhỏ Fermat và cung cấp **cơ sở toán học cốt lõi cho quá trình giải mã RSA**. Nếu các số mũ e và d thỏa mãn

$$ed = k\varphi(N) + 1$$

với một số nguyên k , thì

$$x^{ed} = x^{k\varphi(N)+1} = (x^{\varphi(N)})^k x \equiv x \pmod{N},$$

giả sử $\gcd(x, N) = 1$. Đây là ý tưởng trung tâm đằng sau tính đúng đắn của hoán vị của sập RSA.

2.4 Bài toán logarit rời rạc

Hệ mã ElGamal được xây dựng trên độ khó của **bài toán logarit rời rạc** trong một nhóm cyclic phù hợp. Gọi G là một nhóm cyclic sinh bởi phần tử g . Cho phần tử

$$h = g^a,$$

bài toán logarit rời rạc yêu cầu tìm lại số mũ a . Mặc dù phép lũy thừa có thể tính hiệu quả, phép đảo ngược của nó được tin là khó về mặt tính toán trong các nhóm được lựa chọn cẩn thận và có cấp lớn.

Sự bất đối xứng giữa phép tính thuận dễ và phép đảo ngược khó chính là khuôn mẫu tổng quát xuất hiện xuyên suốt trong mã hóa khóa công khai. Trong ElGamal, khóa công khai chứa một phần tử nhóm có dạng g^a , trong khi khóa bí mật chính là số mũ a . Mức độ an toàn phụ thuộc vào giả thuyết rằng đối thủ không thể khôi phục a một cách hiệu quả chỉ từ thông tin công khai.

2.5 Giả thuyết Diffie–Hellman tính toán

Giả thuyết Computational Diffie–Hellman (CDH) mô tả hình thức độ khó của việc kết hợp các phần tử nhóm đã được nâng lên lũy thừa thành một bí mật chung. Cho

$$g, \quad g^a, \quad g^b,$$

yêu cầu là tính

$$g^{ab}.$$

Giả thuyết CDH phát biểu rằng không có thuật toán hiệu quả nào có thể thực hiện phép tính này với xác suất không bỏ qua được trong các nhóm được dùng bởi hệ mã.

Bài toán này liên quan mật thiết đến bài toán logarit rời rạc, nhưng không hoàn toàn đồng nhất với nó. Ngay cả khi đối thủ không thể tìm ra riêng lẻ a hoặc b , câu hỏi vẫn còn đó là liệu giá trị chung g^{ab} có thể được tính trực tiếp hay không. Trong nhiều bối cảnh mật mã, CDH được xem là một giả thuyết độ khó tự nhiên cho thỏa thuận khóa và các cấu trúc liên quan.

2.6 Giả thuyết Diffie–Hellman quyết định

Giả thuyết Decisional Diffie–Hellman (DDH) mạnh hơn CDH và đặc biệt quan trọng đối với an toàn của mã hóa ElGamal. Cho bộ

$$(g, g^a, g^b, T),$$

thách thức là quyết định xem

$$T = g^{ab}$$

hay T chỉ đơn giản là một phần tử ngẫu nhiên của nhóm, độc lập với a và b . Dưới giả thuyết DDH, không có đối thủ hiệu quả nào có thể phân biệt hai trường hợp này với lợi thế đáng kể.

Dạng phát biểu theo kiểu “quyết định” này quan trọng bởi vì tính bảo mật ngữ nghĩa yêu cầu các thành phần của bản mã phải có vẻ ngẫu nhiên đối với đối thủ. Trong ElGamal, giá trị $h^r = g^{ar}$ được ẩn trong bản mã. Nếu bộ gồm các giá trị công khai và thành phần ẩn này không thể phân biệt được với ngẫu nhiên bằng tính toán, thì thông điệp được mã hóa cũng được bảo vệ trước đối thủ hiệu quả.

2.7 Giả thuyết phân tích thừa số

RSA được xây dựng trên một giả thuyết độ khó khác. Thay vì dựa vào logarit rời rạc trong nhóm, nó dựa vào độ khó được giả định của việc phân tích một số thành lớn

$$N = pq,$$

trong đó p và q là các số nguyên tố lớn. Giả thuyết phân tích thừa số phát biểu rằng, khi chỉ biết N , không có thuật toán hiệu quả nào có thể tìm lại các thừa số nguyên tố của nó nếu mô-đun được chọn đủ lớn và sinh ra đúng cách.

Giả thuyết phân tích thừa số này quan trọng bởi vì biết được p và q sẽ lập tức cho phép suy ra

$$\varphi(N) = (p - 1)(q - 1),$$

và từ đó cho phép tính số mũ bí mật d từ số mũ công khai e . Vì vậy, độ an toàn của RSA gắn liền với độ khó của việc khôi phục phép phân tích thừa số ẩn của N .

Tổng hợp lại, các khái niệm toán học này tạo nên nền tảng cho hai hệ thống chính được nghiên cứu trong chương này. **ElGamal** dựa vào các giả thuyết độ khó liên quan đến lũy thừa trong nhóm cyclic, đặc biệt là DDH, trong khi **RSA** dựa vào số học modulo một số hợp thành và độ khó của việc phân tích mô-đun đó.

3. ElGamal Cryptosystem

ElGamal Cryptosystem là một public-key encryption scheme được xây dựng từ phép lũy thừa trong một nhóm cyclic. Độ an toàn của nó gắn với độ khó của việc phân biệt các bộ Diffie–Hellman với các phần tử ngẫu nhiên. Trong cách trình bày của chương này, thông điệp được bảo vệ bằng cách kết hợp một mặt nạ sinh từ nhóm với bản rõ, nên bản mã sẽ thay đổi mỗi khi cùng một thông điệp được mã hóa.

3.1 Sinh khóa

Gọi G là một nhóm cyclic cấp nguyên tố q được sinh bởi phần tử g . Người nhận chọn một số mũ bí mật

$$a \xleftarrow{\$} \{1, 2, \dots, q - 1\}$$

và tính

$$h = g^a.$$

Khóa công khai là

$$pk = (G, g, h),$$

và khóa bí mật là

$$sk = a.$$

Giá trị h được công khai, nhưng việc khôi phục a từ g và g^a được tin là khó trong một nhóm thích hợp. Đây là cấu trúc một chiều cơ bản đứng sau ElGamal.

3.2 Mã hóa

Để mã hóa một thông điệp m , người gửi chọn một số mũ ngẫu nhiên mới

$$r \xleftarrow{\$} \{1, 2, \dots, q-1\}.$$

Bản mã sau đó được tạo thành dưới dạng

$$C = (u, v) = (g^r, h^r \cdot m).$$

Thành phần thứ nhất $u = g^r$ biểu diễn giá trị ngẫu nhiên dưới dạng lũy thừa, còn thành phần thứ hai che thông điệp bằng cách nhân nó với giá trị chung $h^r = g^{ar}$. Đặc điểm quan trọng là r được chọn mới cho mỗi lần mã hóa, nên ngay cả cùng một bản rõ cũng tạo ra những bản mã khác nhau.

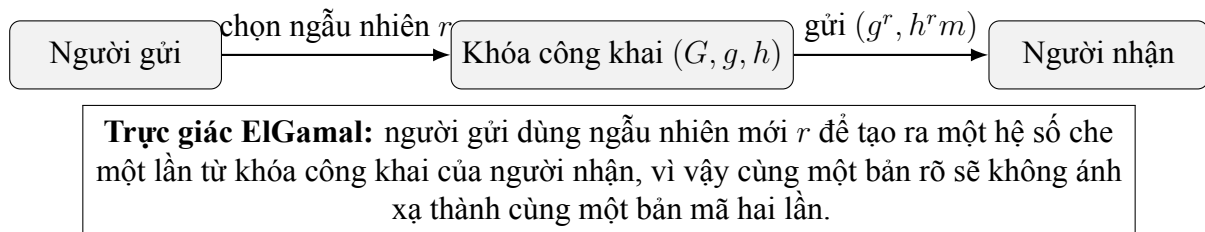


Figure 2: Luồng mã hóa ElGamal ở mức khái quát.

3.3 Giải mã

Cho bản mã

$$(u, v) = (g^r, h^r \cdot m),$$

người nhận dùng khóa bí mật a để tính

$$u^a = (g^r)^a = g^{ra} = h^r.$$

Thông điệp sau đó được khôi phục bằng cách loại bỏ mặt nạ:

$$m = \frac{v}{u^a}.$$

Vì

$$\frac{h^r \cdot m}{(g^r)^a} = \frac{g^{ar} \cdot m}{g^{ar}} = m,$$

nên quá trình giải mã là đúng.

3.4 Phân tích an toàn

Mệnh đề an toàn quan trọng ở đây là ElGamal đạt **semantic security** dưới **giả thuyết Decisional Diffie–Hellman**. Cấu trúc của bản mã là

$$(g^r, h^r \cdot m) = (g^r, g^{ar} \cdot m).$$

Nếu một đối thủ có thể phân biệt các phép mã hóa của hai thông điệp được chọn, thì đối thủ đó có thể quyết định xem một bộ dạng

$$(g, g^a, g^r, T)$$

có chứa

$$T = g^{ar}$$

hay chỉ chứa một phần tử ngẫu nhiên của nhóm. Điều này đưa đến ý tưởng phép quy dẫn:

Phá semantic security của ElGamal $\implies$ phân biệt được một bộ DDH.

Trực giác của lập luận này khá rõ ràng. Nếu $T = g^{ar}$ thì bản mã thử thách là một phép mã hóa ElGamal hợp lệ. Nếu T là ngẫu nhiên, thì thành phần che hoạt động giống như một one-time pad trong bối cảnh nhóm, và đối thủ không thu được thông tin hữu ích nào về thông điệp. Do đó, bất kỳ thành công không bỏ qua được nào trong việc phân biệt thông điệp đều sẽ mâu thuẫn với giả thuyết DDH.

3.5 Ví dụ tính toán nhỏ

Để minh họa, xét một ví dụ nhỏ đủ đơn giản để kiểm tra bằng tay. Lấy nhóm nhân modulo 23, và chọn phần tử sinh

$$g = 5.$$

Giả sử người nhận chọn

$$a = 6.$$

Khi đó

$$h = g^a = 5^6 \equiv 8 \pmod{23},$$

vì vậy khóa công khai là $(23, 5, 8)$ và khóa bí mật là 6.

Bây giờ, giả sử thông điệp là

$$m = 10,$$

và người gửi chọn ngẫu nhiên

$$r = 7.$$

Người gửi tính

$$u = g^r = 5^7 \equiv 17 \pmod{23}$$

và

$$h^r = 8^7 \equiv 12 \pmod{23}.$$

Do đó, thành phần bản mã thứ hai là

$$v = h^r \cdot m \equiv 12 \cdot 10 \equiv 5 \pmod{23}.$$

Vì vậy bản mã thu được là

$$C = (17, 5).$$

Để giải mã, người nhận tính

$$u^a = 17^6 \equiv 12 \pmod{23}.$$

Nghịch đảo của 12 modulo 23 là 2, vì

$$12 \cdot 2 \equiv 1 \pmod{23}.$$

Từ đó

$$m = v \cdot (u^a)^{-1} \equiv 5 \cdot 2 \equiv 10 \pmod{23},$$

và thông điệp gốc được khôi phục.

Ví dụ này cố ý sử dụng tham số nhỏ và không an toàn trong thực tế, nhưng nó cho thấy rõ ba giai đoạn cốt lõi của ElGamal: **sinh khóa**, **mã hóa ngẫu nhiên**, và **loại bỏ mặt nạ thông qua phép lũy thừa với khóa bí mật**.

4. RSA Trapdoor Permutation

RSA được hiểu tự nhiên nhất như một **trapdoor permutation** trên nhóm nhân modulo một số hợp thành. Khóa công khai xác định một hàm dễ tính theo chiều thuận, còn khóa bí mật cung cấp thông tin ẩn cần thiết để đảo ngược hàm đó một cách hiệu quả.

4.1 Sinh khóa

RSA bắt đầu bằng việc chọn hai số nguyên tố lớn phân biệt

$$p \quad \text{và} \quad q,$$

và hình thành mô-đun

$$N = pq.$$

Hàm phi Euler của N là

$$\varphi(N) = (p-1)(q-1).$$

Tiếp theo, ta chọn số mũ công khai e sao cho

$$\gcd(e, \varphi(N)) = 1.$$

Vì e nguyên tố cùng nhau với $\varphi(N)$, nó có nghịch đảo nhân modulo $\varphi(N)$. Số mũ bí mật d được xác định bởi

$$ed \equiv 1 \pmod{\varphi(N)}.$$

Tương đương, tồn tại một số nguyên k sao cho

$$ed = k\varphi(N) + 1.$$

Khóa công khai là

$$pk = (N, e),$$

và khóa bí mật là

$$sk = d.$$

Phép phân tích thừa số của N không được công bố. Chính phép phân tích ẩn này cho phép tính được số mũ bí mật ngay từ đầu.

4.2 Mã hóa

Với một thông điệp M thuộc nhóm nhân $\mathbb{Z}_N^*$, phép mã hóa RSA được định nghĩa bởi

$$C = M^e \bmod N.$$

Phép tính này hiệu quả ngay cả khi N rất lớn, vì lũy thừa modulo có thể được thực hiện bằng kỹ thuật bình phương lặp.

4.3 Giải mã

Giải mã sử dụng số mũ bí mật d :

$$M = C^d \bmod N.$$

Vì $C = M^e$, quá trình giải mã thực chất tính

$$C^d = (M^e)^d = M^{ed}.$$

Do đó, tính đúng đắn của RSA tương đương với việc chứng minh rằng

$$M^{ed} \equiv M \pmod{N}.$$

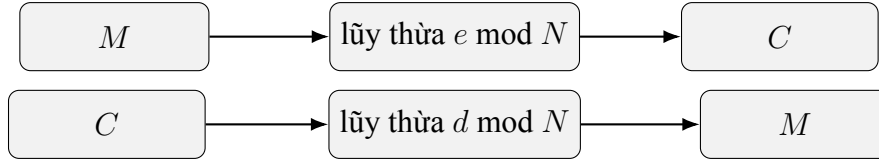


Figure 3: RSA áp dụng số mũ công khai theo chiều thuận và số mũ bí mật theo chiều đảo ngược.

4.4 Giải thích trapdoor permutation

Trapdoor permutation là một song ánh dễ tính theo chiều thuận nhưng khó đảo ngược nếu không có thông tin đặc biệt. RSA phù hợp với mô hình này. Ánh xạ

$$M \mapsto M^e \bmod N$$

là công khai và có thể tính hiệu quả. Tuy nhiên, việc đảo ngược nó tương đương với việc trích xuất căn bậc e modulo N , điều được tin là khó về mặt tính toán nếu không có thông tin cửa sập.

Cửa sập ở đây là số mũ bí mật d , hay tương đương là phép phân tích thừa số của N . Một khi phép phân tích đó được biết, ta có thể tính $\varphi(N)$ và từ đó suy ra d . Nếu thiếu cấu trúc ẩn này, việc đảo ngược được giả định là bất khả thi trong thời gian hiệu quả với các tham số đủ lớn.

4.5 Chứng minh tính đúng đắn

Chứng minh tính đúng đắn của RSA suy ra trực tiếp từ định lý Euler. Vì

$$ed = k\varphi(N) + 1,$$

ta có

$$M^{ed} = M^{k\varphi(N)+1} = (M^{\varphi(N)})^k \cdot M.$$

Nếu $M \in \mathbb{Z}_N^*$, thì $\gcd(M, N) = 1$, nên theo định lý Euler,

$$M^{\varphi(N)} \equiv 1 \pmod{N}.$$

Thay vào biểu thức trên, ta được

$$M^{ed} \equiv 1^k \cdot M \equiv M \pmod{N}.$$

Do đó,

$$C^d \equiv (M^e)^d \equiv M \pmod{N},$$

và việc giải mã thực sự thu lại đúng thông điệp gốc.

Chứng minh này quan trọng vì nó cho thấy RSA không đơn thuần là một mẹo thuật toán. Tính đúng đắn của nó bắt nguồn từ cấu trúc của lý thuyết số, cụ thể là mối quan hệ giữa e và d theo modulo $\varphi(N)$.

5. Vì sao Textbook RSA không an toàn

Mặc dù RSA cung cấp một trapdoor permutation rất đẹp về mặt toán học, **textbook RSA không phải là một sơ đồ mã hóa an toàn**. Hàm

$$E(M) = M^e \bmod N$$

là tất định và có cấu trúc đại số rõ rệt. Chính những tính chất này khiến nó dễ bị nhiều kiểu tấn công vi phạm bảo mật ngữ nghĩa và cho phép thao tác có chủ đích lên bản mã.

5.1 Mã hóa tất định

Trong một sơ đồ mã hóa an toàn, việc mã hóa cùng một thông điệp hai lần không nên để lộ rằng các bản rõ giống nhau. RSA giáo khoa thất bại ở yêu cầu này vì nó là tất định:

$$E(M) = M^e \bmod N$$

chỉ có đúng một đầu ra cho mỗi thông điệp M . Do đó, nếu đối thủ quan sát hai bản mã

$$C_1 = E(M) \quad \text{và} \quad C_2 = E(M),$$

thì tất yếu

$$C_1 = C_2.$$

Điều này ngay lập tức làm rò rỉ thông tin về tính bằng nhau. Đối thủ cũng có thể thực hiện tấn công từ điển đối với những thông điệp có thể dự đoán bằng cách tự mã hóa các ứng viên dưới khóa công khai rồi so sánh với bản mã quan sát được. Chỉ riêng lý do này đã đủ cho thấy textbook RSA **không đạt semantic security**.

5.2 Tấn công thông điệp nhỏ

Một điểm yếu khác xuất hiện khi bản rõ có giá trị số nhỏ. Giả sử

$$M^e < N.$$

Khi đó phép giảm modulo thực ra không làm thay đổi giá trị, và bản mã đơn giản là

$$C = M^e$$

trong số học thông thường. Khi ấy, đối thủ có thể khôi phục thông điệp bằng cách lấy căn bậc e trong số nguyên:

$$M = \sqrt[e]{C}.$$

Kiểu tấn công này đặc biệt nguy hiểm khi số mũ công khai e nhỏ và không gian bản rõ chứa

các thông điệp ngắn hoặc có cấu trúc. Nó cho thấy tính đúng đắn đại số không đồng nghĩa với tính bí mật mật mã.

5.3 Tấn công dựa trên tính nhân

RSA giáo khoa còn bảo toàn phép nhân:

$$E(M_1) \cdot E(M_2) \equiv M_1^e M_2^e \equiv (M_1 M_2)^e \equiv E(M_1 M_2) \pmod{N}.$$

Tính chất nhân này có nghĩa là các bản mã có thể bị sửa đổi theo cách có ý nghĩa mà không cần biết bản rõ tương ứng. Nếu đối thủ nắm giữ bản mã $C = E(M)$ và chọn một giá trị t , thì

$$C' = C \cdot E(t) \equiv E(Mt) \pmod{N}$$

là một bản mã hợp lệ của bản rõ liên hệ Mt .

Điều này phá vỡ trực giác cho rằng dữ liệu đã mã hóa cần chống lại các thao tác đại số có chủ đích. Ngay cả khi đối thủ không khôi phục hoàn toàn được thông điệp, việc có thể biến đổi bản mã thành một bản mã khác với ý nghĩa có thể dự đoán được đã là một thất bại nghiêm trọng về an toàn.

5.4 Ví dụ tấn công trong bài giảng

Ví dụ trong bài giảng minh họa cách một khóa phiên ngắn được mã hóa bằng RSA có thể bị tấn công nhanh hơn rất nhiều so với vét cạn. Giả sử trình duyệt mã hóa một khóa phiên 64 bit K bằng RSA giáo khoa:

$$C = K^e \bmod N.$$

Giả sử thêm rằng

$$K = K_1 K_2$$

với

$$K_1, K_2 < 2^{34}.$$

Khi đó

$$C \equiv K_1^e K_2^e \pmod{N}.$$

Đối thủ có thể dùng chiến lược meet-in-the-middle:

1. Tiền tính một bảng các giá trị từ một phía, chẳng hạn

$$\frac{C}{K_1^e} \pmod{N}$$

cho mọi ứng viên của K_1 .

2. Tính

$$K_2^e \pmod{N}$$

cho mọi ứng viên của K_2 và kiểm tra xem có giá trị nào khớp trong bảng hay không.

Chiến lược này giảm đáng kể khối lượng tính toán so với việc vét cạn trực tiếp toàn bộ 2^{64} khả năng. Thay vì duyệt toàn bộ không gian khóa, đối thủ khai thác cấu trúc nhân của RSA để tách bài toán thành hai tìm kiếm nhỏ hơn.

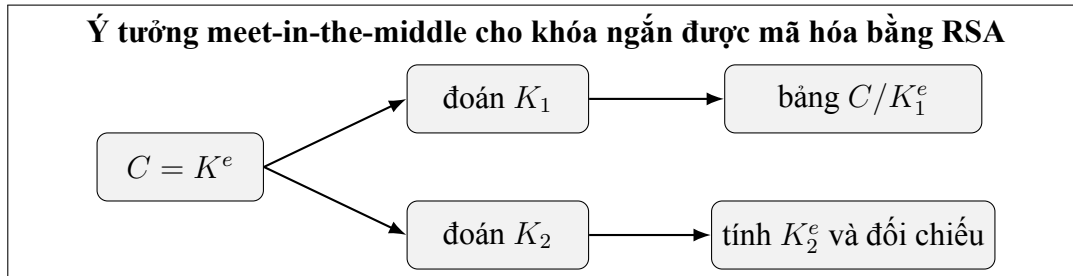


Figure 4: Một không gian bản rõ có cấu trúc có thể khiến RSA giáo khoa bị tấn công nhanh hơn vét cạn.

Những tấn công này minh họa bài học trung tâm của phần này: **RSA với tư cách là một trapdoor permutation không tự động trở thành một sơ đồ mã hóa khóa công khai an toàn.** Cần có thêm bước tiền xử lý thích hợp để đưa ngẫu nhiên vào và loại bỏ cấu trúc đại số có thể bị khai thác.

6. RSA Padding và mã hóa RSA an toàn

Những điểm yếu của textbook RSA xuất hiện vì hàm thô

$$M \mapsto M^e \pmod{N}$$

là tắt định và có cấu trúc đại số rõ ràng. Để biến RSA thành một sơ đồ mã hóa thực tế, trước hết cần biến đổi thông điệp bằng một quá trình mã hóa tiền xử lý ngẫu nhiên. Quá trình này được gọi là **padding** (đệm).

6.1 Vì sao cần đệm

Padding phục vụ hai mục tiêu thiết yếu. Thứ nhất, nó đưa vào **tính ngẫu nhiên**, bảo đảm rằng cùng một thông điệp không luôn luôn cho ra cùng một bản mã. Thứ hai, nó thêm **cấu trúc**, để những đầu vào sai định dạng hoặc bị sửa đổi có thể được phát hiện và từ chối sau khi giải mã.

Về mặt khái niệm, đệm biến quá trình mã hóa từ

$$C = M^e \pmod{N}$$

thành

$$C = \text{RSA}(\text{Pad}(M; r)) = \text{Pad}(M; r)^e \bmod N,$$

trong đó r là ngẫu nhiên mới, được lấy độc lập cho mỗi lần mã hóa. Khi hàm RSA được áp dụng lên một biểu diễn đã ngẫu nhiên hóa của thông điệp thay vì lên chính thông điệp, tính tất định và cấu trúc nhân từng bị khai thác chống lại textbook RSA không còn dẫn tới các tấn công hữu dụng: kiểm tra bằng nhau, tấn công từ điển, khai căn với thông điệp nhỏ và thao tác bản mã đều trở nên khó hơn rất nhiều.

Một cách phát biểu mục tiêu thiết kế là padding phải biến RSA thành một sơ đồ **an toàn ngữ nghĩa** (semantic security), và lý tưởng là an toàn trước **tấn công bản mã được chọn thích nghi** (IND-CCA2), sao cho đối phương không học được gì từ một bản mã ngay cả khi được phép truy vấn một oracle giải mã trên các bản mã khác.

6.2 PKCS#1 v1.5

Một sơ đồ đệm có ý nghĩa lịch sử quan trọng là **PKCS#1 v1.5**. Gọi k là độ dài tính bằng byte của mô-đun N . Trước khi mã hóa RSA, một thông điệp M gồm mLen byte được nhúng vào một khối k byte có dạng

$$\underbrace{00}_1 \parallel \underbrace{02}_1 \parallel \underbrace{\text{PS}}_{\geq 8} \parallel \underbrace{00}_1 \parallel \underbrace{M}_{\text{mLen}}.$$

Các thành phần có vai trò cụ thể:

- byte 00 đứng đầu bảo đảm rằng khối, khi đọc như một số nguyên, nhỏ hơn N ;
- byte kiểu khối 02 cho biết khối này dành cho *mã hóa*;
- chuỗi đệm PS gồm các byte ngẫu nhiên **khác 0** và phải dài ít nhất 8 byte, đây chính là phần cung cấp tính ngẫu nhiên;
- byte phân cách 00 đánh dấu nơi kết thúc phần đệm và bắt đầu thông điệp.

Vì PS phải chiếm ít nhất 8 byte và các byte cố định chiếm thêm 3 byte, độ dài thông điệp bị chặn bởi $\text{mLen} \leq k - 11$. Để giải mã, ta tính $C^d \bmod N$, biểu diễn lại thành khối k byte, rồi kiểm tra tiền tố 00 02, vùng đệm khác 0 và byte phân cách 00; thông điệp là phần đứng sau đó.

Thiết kế này từng được triển khai rộng rãi và là một bước tiến lớn so với RSA giáo khoa. Tuy nhiên, về sau người ta chứng minh nó có thể bị tấn công bản mã được chọn thích nghi mỗi khi một cài đặt làm lộ việc khối sau giải mã có đúng định dạng PKCS#1 hay không (xem Mục 7). Vì vậy, PKCS#1 v1.5 minh họa một chủ đề lặp đi lặp lại trong mật mã: một quy tắc mã hóa trông mạnh trên giấy vẫn có thể thất bại khi kết hợp với một cài đặt làm rò rỉ dù chỉ một bit về giá trị sau giải mã.

6.3 OAEP

Một thiết kế hiện đại và có cơ sở lý thuyết hơn là **Optimal Asymmetric Encryption Padding (OAEP)**, do Bellare và Rogaway đề xuất. OAEP tiền xử lý thông điệp bằng ngẫu nhiên mới kết hợp với hai hàm sinh mật dựa trên hàm băm, ký hiệu G và H , trước khi áp dụng phép lũy thừa RSA. Ở mức khái quát, nó:

- thêm một hạt giống ngẫu nhiên mới cho mỗi lần mã hóa,
- trộn thông điệp và hạt giống qua một mạng kiểu Feistel hai vòng dựng từ G và H , và
- tạo ra một khối mã hóa có tính *tất cả hoặc không gì*: chỉ có thể khôi phục thông điệp từ toàn bộ khối, không bao giờ từ một phần của nó.

Cụ thể, gọi $hLen$ là độ dài đầu ra của hàm băm và k là độ dài byte của N . Thông điệp trước hết được định dạng thành một khối dữ liệu

$$DB = lHash \parallel PS \parallel 01 \parallel M,$$

trong đó $lHash$ là băm của một nhãn tùy chọn và PS là một dãy các byte 0. Sau đó lấy một hạt giống ngẫu nhiên dài $hLen$, và thông điệp đã mã hóa được tính như sau:

$$\begin{aligned} \text{maskedDB} &= DB \oplus G(\text{seed}), \\ \text{maskedSeed} &= \text{seed} \oplus H(\text{maskedDB}), \\ EM &= 00 \parallel \text{maskedSeed} \parallel \text{maskedDB}, \end{aligned}$$

và EM là số nguyên được đưa vào phép hoán vị RSA. Giải mã chỉ việc đảo ngược hai vòng: H khôi phục hạt giống từ maskedSeed và maskedDB , rồi G khôi phục DB , sau đó kiểm tra định dạng của DB .

OAEP quan trọng vì nó đi kèm các bảo đảm lý thuyết mạnh hơn. Cấu trúc này có tính *tất cả hoặc không gì* (all-or-nothing): lật một bit bất kỳ của EM sẽ làm xáo trộn không đoán trước được cả hạt giống lẫn khối dữ liệu khôi phục, nên kẻ tấn công không thể biến một bản mã hợp lệ thành một bản mã hợp lệ khác. Trong mô hình random oracle, RSA-OAEP được chứng minh là **an toàn IND-CCA2** dưới giả thuyết RSA, tức chính khả năng chống tấn công bản mã được chọn thích nghi mà PKCS#1 v1.5 còn thiếu.

Bài học rộng hơn là **mã hóa RSA an toàn không chỉ đơn giản là “RSA cộng một thông điệp”**. Nó là phép hoán vị RSA hợp thành với một thủ tục mã hóa ngẫu nhiên được thiết kế cẩn thận. Nếu thiếu bước tiền xử lý này, trapdoor permutation vẫn thanh nhả về toán học nhưng không an toàn về mật mã.

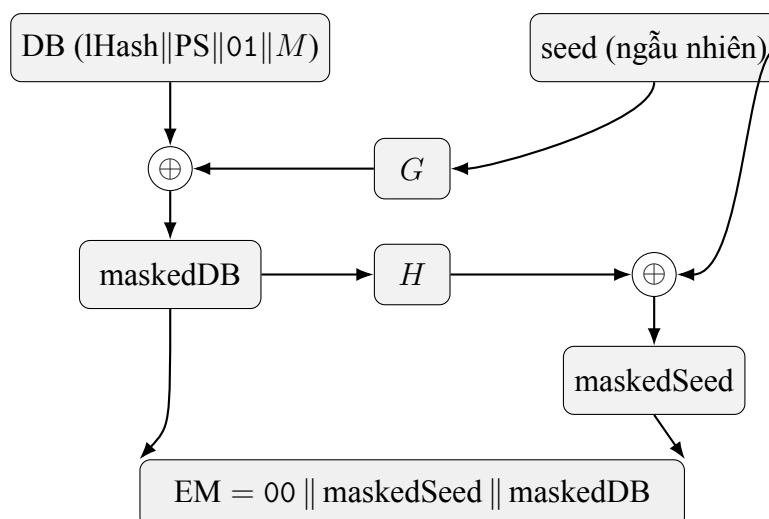


Figure 5: OAEP kết hợp khối dữ liệu DB và một hạt giống ngẫu nhiên qua hai hàm sinh mật mã G và H trước khi áp dụng RSA. Giải mã phải xử lý toàn bộ khối: muốn khôi phục M cần toàn bộ maskedDB (để dựng lại hạt giống qua H) và toàn bộ hạt giống (để dựng lại DB qua G).

7. Tình huống tấn công thực tế: Tấn công Bleichenbacher

Một trong những tấn công thực tế có ảnh hưởng sâu rộng nhất đối với mã hóa RSA là **tấn công Bleichenbacher**, do Daniel Bleichenbacher công bố năm 1998. Nó nhắm vào các cài đặt RSA dùng đệm PKCS#1 v1.5 — bao gồm cả trao đổi khóa RSA trong SSL/TLS — và cho thấy rằng ngay cả một nguyên thủy mạnh về toán học cũng có thể thất bại khi cài đặt làm lộ chỉ một bit thông tin về dữ liệu sau giải mã.

7.1 Bối cảnh

PKCS#1 v1.5 từng được dùng rộng rãi trong các giao thức mạng và phần mềm thời kỳ đầu. Như đã mô tả ở Mục 6, một khối hợp lệ dành cho mã hóa phải bắt đầu bằng các byte 00 02, theo sau là ít nhất tám byte đệm khác 0, một byte phân cách 00 và thông điệp. Máy chủ thường kiểm tra xem khối sau giải mã có thỏa định dạng này không trước khi tiếp tục giao thức. Nếu phần đệm không hợp lệ, cài đặt thường dừng với lỗi hoặc có hành vi quan sát được khác với trường hợp hợp lệ.

Chính sự khác biệt hành vi này mở đường cho tấn công Bleichenbacher. Kẻ tấn công không cần khóa bí mật, cũng không cần phá trực tiếp RSA. Thay vào đó, họ khai thác phản ứng của máy chủ trước những bản mã được sửa đổi có chủ đích.

7.2 Padding oracle

Tấn công dựa trên một **padding oracle**: bất kỳ cơ chế nào cho kẻ tấn công biết liệu bản rõ sau giải mã có thỏa định dạng PKCS#1 yêu cầu hay không. Ví dụ:

- đệm hợp lệ $\rightarrow$ máy chủ tiếp tục xử lý (hoặc phản hồi chậm hơn, hoặc trả về một lỗi khác),
- đệm không hợp lệ $\rightarrow$ máy chủ báo lỗi hoặc phản hồi khác đi.

Ngay cả một bit thông tin như vậy cũng đủ nguy hiểm. Vì một khối hợp lệ bắt đầu bằng 00 02, câu trả lời “đệm hợp lệ” cho kẻ tấn công biết số nguyên sau giải mã nằm trong một khoảng hẹp và đoán trước được, và mỗi truy vấn lại tinh chỉnh hiểu biết của kẻ tấn công về bản rõ ẩn.

7.3 Diễn tiến tấn công

Gọi k là độ dài byte của N và đặt

$$B = 2^{8(k-2)}.$$

Một bản rõ m có khối mã hóa bắt đầu bằng các byte 00 02 thỏa, khi xét như số nguyên,

$$2B \leq m \leq 3B - 1.$$

Đây chính là khoảng mà một câu trả lời oracle dương tiết lộ.

Giả sử kẻ tấn công chặn được bản mã mục tiêu

$$C = M^e \bmod N.$$

Kẻ tấn công chọn một hệ số s và tạo bản mã sửa đổi

$$C' = C \cdot s^e \bmod N.$$

Theo tính chất nhân của RSA, khi máy chủ giải mã C' nó thu được

$$M' = Ms \bmod N.$$

Oracle khi đó cho biết M' có hợp lệ PKCS#1 hay không. Cụ thể, một M' hợp lệ bắt đầu bằng các byte 00 02, nên

$$2B \leq Ms - rN \leq 3B - 1 \quad \text{với một số nguyên } r \text{ nào đó.}$$

(Hợp lệ đầy đủ còn đòi hỏi phần đệm khác 0 và một byte phân cách 00, nên khoảng này là điều kiện lỏng hơn một chút so với phép kiểm tra đầy đủ; phần lập luận khoảng dưới đây dùng tiền tố 00 02, như trong phân tích gốc của Bleichenbacher.) Mỗi giá trị s hợp lệ cho một ràng buộc tuyến tính dạng này, và do đó dồn M về một hợp của các khoảng con của $[2B, 3B - 1]$. Bằng cách tìm các hệ số s kế tiếp một cách có hệ thống và giao các ràng buộc thu được, kẻ tấn công liên tục thu nhỏ tập giá trị khả dĩ cho đến khi chỉ còn một khoảng rộng bằng một; điểm nút của nó chính là bản rõ gốc M . Vì vậy đây không phải là phép đảo RSA trực tiếp bằng đại số mà là một **quá trình tìm kiếm thu hẹp khoảng thích nghi** dựa trên phản hồi oracle, thường cần cỡ hàng chục nghìn đến vài triệu truy vấn tùy biến thể.

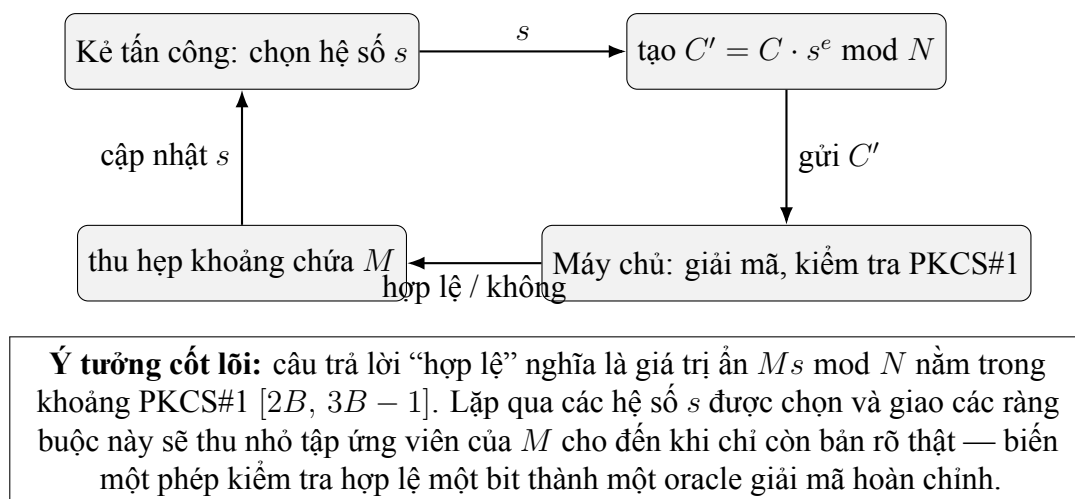


Figure 6: Tấn công Bleichenbacher là một vòng lặp phản hồi thích nghi: mỗi bit oracle thu hẹp khoảng phải chứa bản rõ.

7.4 Vì sao điều này quan trọng

Tấn công Bleichenbacher thay đổi cách người ta nhìn nhận an toàn của hệ mã. Bản thân hàm RSA không bị phá. Sự thất bại đến từ tương tác giữa:

- cấu trúc đệm định nghĩa một định dạng “hợp lệ” nhận biết được,
- một cài đặt có kiểm tra định dạng đó, và
- một khác biệt hành vi quan sát được từ bên ngoài giữa trường hợp hợp lệ và không hợp lệ.

Đây là một bài học mạnh mẽ. Một hệ thống có thể được dựng từ những thành phần toán học rất mạnh nhưng vẫn mất an toàn nếu cài đặt làm rò rỉ thông tin từng phần. Tấn công này còn tỏ ra bền bỉ đáng kể: gần hai thập kỷ sau, các biến thể như **DROWN** (2016, khai thác SSLv2 cũ) và **ROBOT** — “Return Of Bleichenbacher’s Oracle Threat” (2017) — cho thấy cùng oracle đó vẫn tồn tại trong các thư viện TLS được triển khai rộng rãi, vì PKCS#1 v1.5 rất khó cài đặt mà không để lộ một kênh phụ về thời gian hoặc thông báo lỗi.

Vì lý do đó, tấn công Bleichenbacher không chỉ là một ví dụ lịch sử. Nó là minh họa kinh điển cho việc **rò rỉ ở mức cài đặt có thể phá hủy hoàn toàn một nguyên thủy vốn mạnh về toán học**, và nó đã thúc đẩy mạnh mẽ việc chuyển sang các cách mã hóa chứng minh được an toàn CCA như OAEP, cũng như sang các cài đặt thời gian hằng và chống rò rỉ nói chung.

8. Kết luận

Mã hóa khóa công khai giải quyết một bài toán mà mã hóa đối xứng đơn lẻ xử lý chưa thuận tiện: thiết lập khóa an toàn giữa các bên chưa chia sẻ bí mật từ trước. Chương này khảo sát hai cấu trúc lớn đạt tới mục tiêu đó trên những nền tảng toán học khác nhau.

Cấu trúc thứ nhất, **ElGamal**, dựa trên phép lũy thừa trong một nhóm cyclic và nhận độ an toàn từ độ khó của các bài toán kiểu Diffie–Hellman, đặc biệt là **giả thuyết Decisional Diffie–Hellman**. Việc dùng ngẫu nhiên mới trong mỗi lần mã hóa là yếu tố thiết yếu: chính thành phần ngẫu nhiên này ngăn bản mã dễ lộ tính bằng nhau giữa các thông điệp và làm cho semantic security trở nên khả thi.

Cấu trúc thứ hai, **RSA**, được xây dựng từ một trapdoor permutation trên số học modulo một số hợp thành. Tính đúng đắn của nó suy ra từ định lý Euler, còn an toàn thực tiễn gắn với độ khó của việc phân tích thừa số mô-đun. RSA cho thấy cách một cấu trúc lý thuyết số thanh nhã có thể tạo ra một nguyên thủy khóa công khai hiệu quả với chiều thuận rõ ràng và chiều đảo được che giấu.

Đồng thời, chương này cũng chỉ ra rằng **textbook RSA là không an toàn**. Tính tất định, điểm yếu trước thông điệp nhỏ và cấu trúc nhân của hàm thô đều ngăn nó trở thành một sơ đồ mã hóa an toàn nếu đứng riêng lẻ. Vì vậy, một trapdoor permutation *không* đồng nghĩa với một hệ mã khóa công khai an toàn.

Để đạt an toàn trong thực tế, RSA phải được kết hợp với **padding**. Padding đưa ngẫu nhiên và tiền xử lý có cấu trúc vào trước phép lũy thừa, loại bỏ những điểm yếu rõ ràng nhất của sơ đồ giáo khoa. Cách mã hóa cũ hơn là **PKCS#1 v1.5** đã cải thiện đáng kể tình hình, nhưng lịch sử cho thấy nó vẫn có thể thất bại dưới các điều kiện tấn công bản mã được chọn thích nghi. Những thiết kế hiện đại hơn như **OAEP** được đưa ra để khắc phục khoảng trống đó, cung cấp an toàn IND-CCA2 chứng minh được trong mô hình random oracle.

Tấn công Bleichenbacher là minh họa thực tế rõ ràng nhất cho nguyên lý này, và các hậu duệ bền bỉ của nó (DROWN, ROBOT) cho thấy mỗi nguy không phải là một lỗi nhất thời mà là một cạm bẫy mang tính cấu trúc. Tấn công làm sụp đổ các hệ thống đang vận hành không phải bằng cách phá vỡ lỗi toán học, mà bằng cách khai thác một bit thông tin rò rỉ trong quá trình giải mã. Theo nghĩa đó, bài học quan trọng nhất của chương vượt ra ngoài riêng ElGamal hay RSA: **an toàn mật mã phụ thuộc đồng thời vào lý thuyết và cài đặt**. Các giả thuyết độ khó mạnh là cần thiết, nhưng chúng không đủ nếu toàn bộ sơ đồ — cách mã hóa, giao thức và cài đặt — không được thiết kế và triển khai với mức độ cẩn trọng tương xứng.